GameDuo — módulo de pagamentos

Estrutura de pastas em Laravel para a integração com o Asaas

Fala com a API do Asaas Executa sozinho, em fila ou agendador Aplicação comum

```
gameduo/
├── app/
│   ├── Console/Commands/
│   │   ├── LiberarLancamentos.php          vence a carência de 30 dias
│   │   └── ProcessarSaques.php             executa os saques pedidos
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── CheckoutController.php      cliente compra moedas
│   │   │   ├── SaqueController.php         prestador pede o saque
│   │   │   ├── AsaasWebhookController.php  recebe aviso de pagamento
│   │   │   └── AutorizacaoSaqueController.php  aprova a transferência
│   │   └── Middleware/
│   │       └── VerificaTokenAsaas.php      confere o header do webhook
│   ├── Jobs/
│   │   ├── CreditarMoedas.php              credita fora do request
│   │   └── ExecutarSaque.php               um job por saque pedido
│   ├── Models/
│   │   ├── Cobranca.php                    a compra de moedas
│   │   ├── MovimentoMoeda.php              extrato do cliente
│   │   ├── Lancamento.php                  o que o prestador tem a receber
│   │   ├── Saque.php                       o pedido de saque
│   │   └── WebhookRecebido.php             garante idempotência
│   └── Services/
│       └── AsaasService.php                único ponto que chama o Asaas
├── config/
│   └── asaas.php                           lê as chaves do .env
├── database/migrations/
│   ├── ..._create_cobrancas_table.php
│   ├── ..._create_movimentos_moeda_table.php
│   ├── ..._create_lancamentos_table.php
│   ├── ..._create_saques_table.php
│   └── ..._create_webhooks_recebidos_table.php
├── routes/
│   ├── api.php                             checkout, saques e webhooks
│   └── console.php                         agenda os dois comandos
├── tests/Feature/
│   ├── CheckoutTest.php
│   ├── AsaasWebhookTest.php                testa aviso duplicado
│   └── SaqueTest.php                       testa carência e saldo
└── .env                                    chave do Asaas, nunca no Git
```

Fluxo de pagamentos

Da compra de moedas até o Pix na conta do prestador



Todas as rotas do módulo

MÉTODO	ROTA	O QUE FAZ
Compra de moedas — autenticado como cliente		
`POST`	`/api/checkout`	Recebe o pacote escolhido, cria a cobrança no Asaas e devolve o QR Code do Pix ou o link do cartão. Grava a cobrança como pendente.
`GET`	`/api/checkout/{id}`	Consulta o status da cobrança. Serve para a tela de espera enquanto o webhook não chega.
`GET`	`/api/carteira`	Devolve o saldo de moedas do cliente, calculado a partir do extrato.
`GET`	`/api/carteira/extrato`	Lista os movimentos de moeda: compras, contratações e devoluções.
Sessões — autenticado como cliente		
`POST`	`/api/sessoes`	Contrata a sessão e retém as moedas correspondentes. Falha se o saldo for insuficiente.
`POST`	`/api/sessoes/{id}/confirmar`	Cliente confirma que a sessão ocorreu. Gera o lançamento a favor do prestador e inicia a carência.
`POST`	`/api/sessoes/{id}/contestar`	Abre disputa dentro do prazo. Segura o lançamento e envia o caso para a moderação.
Carteira do prestador — autenticado como prestador		
`PUT`	`/api/prestador/chave-pix`	Cadastra ou atualiza a chave Pix e o CPF. Sem isso o saque não sai.
`GET`	`/api/prestador/saldo`	Mostra os três números da carteira: disponível para saque, ainda em carência e já comprometido em saques em andamento.
`GET`	`/api/prestador/lancamentos`	Lista sessão a sessão o que o prestador tem a receber, com a data em que cada valor fica disponível.
Saques — autenticado como prestador		
`POST`	`/api/prestador/saques`	Solicita o saque do saldo disponível para a chave Pix cadastrada. Valida o valor mínimo, reserva o saldo e cria o pedido como pendente.
`GET`	`/api/prestador/saques`	Histórico de saques, com valor, taxa, status e comprovante de cada um.
`GET`	`/api/prestador/saques/{id}`	Acompanha um saque específico. É o que a tela consulta enquanto a transferência não conclui.
`DELETE`	`/api/prestador/saques/{id}`	Cancela o pedido enquanto ele ainda está pendente e devolve o valor ao saldo disponível.
Moderação — autenticado como moderador		
`GET`	`/api/moderacao/disputas`	Fila de sessões contestadas aguardando decisão.
`POST`	`/api/moderacao/disputas/{id}`	Registra a decisão: libera o lançamento ao prestador ou devolve as moedas ao cliente.
`GET`	`/api/moderacao/saques`	Fila de saques pedidos, em andamento e falhados, para acompanhar e reprocessar quando necessário.
Webhooks — chamados pelo Asaas, sem login		
`POST`	`/api/webhooks/asaas`	Recebe o aviso de pagamento. Valida o token, grava o evento, despacha o job de crédito e responde 200 imediatamente.
`POST`	`/api/webhooks/autorizacao-saque`	O Asaas pergunta se a transferência é legítima. Só aprova se houver saque pendente com o mesmo valor e a mesma chave.
Agendador — não é rota, roda pelo cron		
`CLI`	`php artisan lancamentos:liberar`	Roda todo dia e marca como disponíveis os lançamentos que completaram a carência de 30 dias.

CLI	`php artisan saques:processar`	Pega os saques pendentes e despacha um job para cada um, que chama a transferência no Asaas.

Por que cada peça existe

As decisões por trás da estrutura de pastas

As cinco tabelas

`cobrancas`	Uma linha por compra de moedas. Guarda o id do pagamento no Asaas e o *externalReference*, que é o que liga o aviso recebido ao pedido certo.
`movimentos_moeda`	Extrato do cliente: entrada na compra, saída ao contratar a sessão, devolução em caso de reembolso. O saldo é a soma da tabela, nunca um campo editado à mão.
`lancamentos`	Uma linha por sessão concluída, com o valor congelado no momento da prestação. Carrega o status, a data da sessão, o lote e o comprovante. Um índice *unique* na sessão impede que uma sessão vire dois pagamentos.
`saques`	Um pedido de saque por linha: valor, taxa, chave Pix usada, status e id da transferência no Asaas. Enquanto o saque está pendente o valor fica reservado, para o prestador não pedir duas vezes o mesmo dinheiro.
`webhooks_recebidos`	Registra o id de cada aviso do Asaas, também com índice *unique*. Se o mesmo aviso chegar duas vezes, o segundo insert falha no banco e nada é creditado em dobro.

As classes que importam

`AsaasService.php`	Única classe do projeto que faz requisição para o Asaas. Quando algo quebrar, há um arquivo só para investigar — e se um dia trocarem de gateway outra vez, é o único que muda.
`AsaasWebhookController.php`	Valida o token, grava o aviso, despacha o job e responde 200 imediatamente. Não credita nada dentro do request: se der erro no meio, o Asaas considera falha e reenvia o aviso.
`AutorizacaoSaqueController.php`	Endpoint separado, com outra finalidade. O Asaas o chama alguns segundos depois de cada transferência pedida, e a aplicação só aprova se existir lançamento correspondente com o mesmo valor. É a proteção contra uma chave de API vazada.
`SaqueController.php`	Recebe o pedido do prestador. Confere o valor mínimo, verifica o saldo disponível e reserva o valor na mesma transação em que cria o pedido. Reservar na hora é o que impede dois pedidos simultâneos de sacarem o mesmo dinheiro.
`ExecutarSaque.php`	Um job por saque, não um job para a fila inteira. Se o Pix de um falhar, os outros já saíram, e a nova tentativa atinge apenas quem falhou.
`LiberarLancamentos.php`	Roda diariamente e muda para disponível os lançamentos que completaram a carência. É o único ponto que faz saldo aparecer na carteira do prestador.